# Panorama: Unveiling Latent Dependencies for Microservice Autoscaling via Meta-learning

Anonymous Author(s)

## Abstract

Microservice autoscaling presents significant challenges due to complex inter-service dependencies. Existing works primarily leverage *direct service calls* to model the propagation effect of scaling decisions among services. However, the intricate and implicit service interactions (i.e., *latent dependencies*) often go beyond such direct invocations, rendering suboptimal scaling actions and poor adaptability. To address these limitations, we propose Panorama, a novel autoscaling framework that explicitly unveils and exploits the indirect service interactions. Particularly, we identify two fundamental categories of latent dependencies, i.e., *resource interference dependencies* (RIDs), which emerge from shared infrastructure resources, and *execution blocking dependencies* (EBDs), which stem from temporal constraints in the execution logic of business workflows. Panorama employs a hybrid attention mechanism to model these multifaceted service dependencies alongside temporal dynamics. We also design a meta-learning paradigm to enable rapid adaptation to evolving workload patterns and dependency structures without extensive retraining. Evaluation across four representative microservice benchmarks shows that Panorama consistently outperforms state-of-the-art baselines, achieving up to 68% reduction in resource consumption while improving response time by 42%. These results highlight the critical importance of modeling latent dependencies for effective microservice resource management.

## CCS Concepts

**• Computer systems organization → Cloud computing**; **• Software and its engineering → Software performance**.

## Keywords

Microservices, Resource Management, Autoscaling, QoS

**ACM Reference Format:**

Anonymous Author(s). 2026. Panorama: Unveiling Latent Dependencies for Microservice Autoscaling via Meta-learning. In *Proceedings of The IEEE/ACM Automated Software Engineering (ASE '26).* ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

## 1 Introduction

Microservice architectures enable independent scaling of individual services [8, 34, 48]. However, such fine-grained scalability introduces significant complexity when managing resources in dynamic production environments. Rapid resource autoscaling has thus become essential for microservice applications to accommodate workload fluctuations. Optimal autoscaling must be able to guarantee resource provisioning during demand surges, while proactively reclaiming underutilized resources to maintain cost-efficiency [7].

State-of-the-art autoscaling methods [29, 36, 37, 51] leverage deep learning to capture intricate and ever-evolving workload patterns. A key strategy used involves modeling spatial and temporal features between microservices through service dependency graphs. They recognize that effective autoscaling requires understanding both inter-service interactions across the system topology (spatial dimension) and their evolution over time (temporal dimension). By analyzing these dual aspects, existing methods can track workload propagation patterns through dependencies and preemptively mitigate bottlenecks before they degrade end-user performance.

While *direct invocations* stand as the most representative type of service dependency, the intricate inter-service relationships in microservice systems go considerably beyond these explicit call graphs [40, 44, 45, 49]. Existing methods that rely solely on direct invocations to model the cascading effects of their scaling decisions often yield an incomplete and potentially inaccurate view of system-wide dynamics. Recent work, such as DeepScaler [29], attempts to address this gap by adaptively learning an "affinity matrix" to uncover hidden statistical correlations among services. However, this method is seeded by the direct call graphs and its learning process remains implicit. Our investigation (Sec. 4.2.3) reveals that this black-box approach fails to capture many critical latent dependencies, offering limited insight into the specific resource bottlenecks or execution constraints underlying performance degradations.

To overcome these limitations, we propose to explicitly model the latent dependencies among microservices, which aims to reveal service interaction patterns beyond direct call graphs and provide more accurate dependency representations. A key contribution of our work is the identification of two critical categories of latent dependencies, which systematically encapsulate the core mechanisms of indirect service performance interactions.

- *Resource Interference Dependencies (RIDs):* At the *infrastructure layer*, RIDs emerge when microservices deployed on the same physical or virtual host compete for shared resources (e.g., CPU and memory), creating performance interdependencies that transcend direct invocations. While some approaches [12, 52] use system metrics to infer such interactions, they often suffer from high modeling complexity and imprecise results.
- *Execution Blocking Dependencies (EBDs):* At the *application layer*, EBDs manifest when a microservice is delayed or impeded due to its reliance on another service to complete a prerequisite task within the business workflow. For example, a checkout service must complete payment before sending a purchase email (Sec. 2.2.3). Crucially, these blocking dependencies exist even without direct invocations. This illustrates



*ASE '26, Munich, Germany*



ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

how a bottleneck in one service can create cascading EBDs throughout the workflow, hindering other services that depend on its output or state change rather than on a direct request.

Based on the identified latent dependencies, we design PANORAMA, a microservice autoscaling framework for more effective and adaptive resource management. PANORAMA introduces a hybrid attention mechanism that decouples attention computation into specialized heads focusing on distinct dependency types alongside temporal dynamics, capturing multifaceted service relationships. Moreover, we design a meta-learning paradigm for rapid adaptation to evolving workload patterns and dependency structures, avoiding the computational overhead of extensive retraining. This is achieved by identifying distinct adaptation tasks and incorporating spatial positional encodings and real-time dependency information as contextual elements. Through this integrated approach, PANORAMA enables coordinated and unified resource adjustments across the entire microservice application while balancing resource efficiency and service quality. Our comprehensive evaluation shows that PANORAMA consistently outperforms state-of-the-art baselines, achieving up to 68% reduction in resource consumption while improving response time by 42% compared to conventional approaches.

To sum up, this work makes the following major contributions:

- We identify and formalize two critical types of latent dependencies that remain largely unaddressed, i.e., RIDs and EBDs. They capture the complete spectrum of service interactions that influence resource requirements and performance propagation.
- We present PANORAMA, which employs a hybrid attention mechanism with a meta-learning strategy for accurate and adaptive autoscaling. By integrating latent dependencies with direct service invocations, PANORAMA delivers coordinated autoscaling decisions that balance service quality and resource efficiency, while adapting effectively to dynamic service environments.
- We implement PANORAMA on K8s and evaluate it across various benchmarks. The results show that PANORAMA achieves significant improvements over existing methods. Further studies confirm the benefits of latent dependencies toward accurate service autoscaling and PANORAMA's ability to discover them.

# 2 Background and Problem Formulation

## 2.1 Microservice Autoscaling

As a crucial operational mechanism, autoscaling dynamically modulates compute resource provisioning to maintain optimal service performance. It typically operates as a closed-loop control system through container orchestration platforms like Kubernetes (K8s). Autoscaling can be achieved through two complementary mechanisms: *horizontal scaling*, which adjusts the number of service instances (replicas), and *vertical scaling*, which modifies the resource allocation (e.g., CPU or memory) of existing instances. The effectiveness of autoscaling is largely influenced by the inherent complexities of microservice systems, introducing two challenges.

- **Multifaceted service dependencies:** The multifaceted nature of service dependencies, which go beyond simple direct invocations, creates complex and often non-obvious performance relationships. This makes it difficult to anticipate how scaling one service might propagate and impact others. Thus, modeling such relationships requires capturing context-sensitive interactions while accounting for intricate service-state variations.
- **Dynamic environments:** The workload patterns and microservice characteristics are constantly evolving due to user behavior changes, code deployments, and configuration modifications. Achieving an optimal balance between QoS guarantees and resource efficiency requires sophisticated modeling and decision-making capabilities that account for both immediate performance needs and future demand forecasts [6, 7].

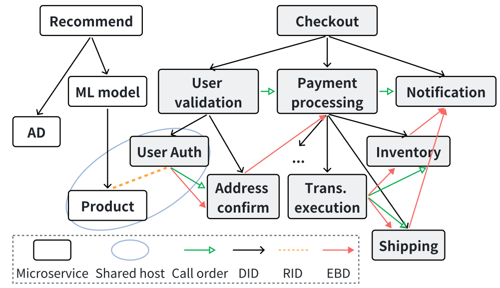


**Figure 1: Heterogeneous Microservice Dependencies**

## 2.2 Heterogeneous Microservce Dependencies

Direct dependencies characterize explicit service interactions but represent only one facet of complex service relationships. We investigate other critical latent dependencies that significantly impact service performance and resource efficiency, but remain unaddressed. We illustrate different dependencies using an e-commence application (partial) in Fig. 1, where the white and gray traces represent the *product recommendation* and *checkout* workflow, respectively.

### 2.2.1 *Direct Invocation Dependencies (DIDs).*

DIDs represent explicit call relationships between microservices, which are relatively straightforward to identify through distributed tracing systems (e.g., Jaeger, Zipkin). The discrepancies in processing throughput or resource allocation among directly connected services may create bottlenecks that amplify latency and degrade end-to-end performance. Thus, many autoscaling approaches [25, 29, 43] incorporate these direct dependencies into their decision-making process, recognizing that effective resource provisioning must consider the entire request execution path rather than local service metrics.

### 2.2.2 *Resource Interference Dependencies (RIDs).*

RIDs represent implicit performance interference between co-located services that compete for shared hardware resources, even without direct communication [21, 36, 45]. While containerization technologies (e.g., cgroups, namespaces, CPU/memory limits) provide OS-level isolation, they cannot fully guarantee performance isolation. Co-located containers still compete for shared hardware resources such as last-level cache (LLC), memory bandwidth, and node-level I/O [21, 45]. Even fine-grained hardware partitioning techniques like Intel Cache Allocation Technology (CAT) or CPU pinning cannot fully eliminate cross-core interference on shared memory buses [12, 52], resulting in unpredictable "noisy neighbor" effects [36, 45]. RIDs are particularly prevalent in cloud environments, where K8s mechanisms like node affinity and anti-affinity rules are soft controls rather than hard guarantees, meaning that scaling operations, node failures, or resource pressure can force co-location of distinct services.

*2.2.3 Execution Blocking Dependencies (EBDs).* EBDs represent implicit performance constraints in hierarchical execution flows and fixed call sequences of microservice workflows. Unlike DIDs, EBDs occur when downstream services experience throttling due to serialized processing requirements with services that must complete earlier in the request lifecycle, even without direct invocations. For the checkout process in Fig. 1, the *checkout* service orchestrates a sequential workflow wherein *user validation* must precede *payment processing*. The *user validation* phase itself involves *user authentication* followed by *address confirmation*. If *address confirmation* experiences bottlenecks, the entire downstream execution chain (e.g., *payment processing*) is blocked regardless of resource availability. A similar blocking dependency exists between the *payment processing* and *notification* service (Sec. 3.3.3). Such dependencies create hidden bottlenecks that conventional methods fail to detect explicitly, as they only analyze call graphs, not end-to-end execution semantics. We formalize these temporal constraints by modeling all potential blocking relationships across service invocation chains, revealing how resource shortages in seemingly unrelated services can cascade through the system via execution ordering requirements.

These execution blocking patterns stem from fundamental aspects of microservice design and operations [19, 27, 35]. Common drivers include synchronous communication protocols (e.g., HTTP, RPC), strict workflow prerequisite tasks, and distributed transaction coordination. Additionally, contention for finite logical resources (e.g., database connection pools) or strict ordering requirements in event-driven pipelines frequently forces downstream services into temporary waiting states.

## 2.3 Problem Formulation

We model a microservice application as a directed graph of multiple interacting services (backed by multiple instances, e.g., Pods in K8s), denoted as $\mathcal{G} = (\mathcal{V}, \mathcal{A})$, where $\mathcal{V} = \{v_1, v_2, \ldots, v_N\}$ is a finite set of nodes representing the services with $|\mathcal{V}| = N$ and $\mathcal{A} = \{\mathcal{A}^{(did)}, \mathcal{A}^{(rid)}, \mathcal{A}^{(ebd)}\}$ is a collection of adjacency matrices. Each matrix $\mathcal{A}^{(*)} \in \{0,1\}^{N\times N}$ represents a distinct dependency type (DID, RID, or EBD) between services and $A_{ij}^{(*)} = 1$ if a dependency exists between $v_i$ and $v_j$. At each time step $t$, we collect a set of $L$ performance metrics for each service $v_i \in \mathcal{V}$ (e.g., CPU utilization, instance count, request rate). These metrics are derived from per-instance measurements, aggregated across all running instances of a service (e.g., summed for request count).

Let $x_{k,i}^t \in \mathbb{R}$ denote the value of the $k$-th metric for service $v_i$ at time $t$, where $k \in \{1, \ldots, L\}$. The vector of metrics for service $v_i$ at time $t$ is $x_i^t = (x_{1,i}^t, x_{2,i}^t, \ldots, x_{L,i}^t)^T \in \mathbb{R}^L$. The state of the entire microservice application at time $t$ is represented by the collection of metrics for all services, denoted by $\mathcal{X}_t = (x_1^t, x_2^t, \ldots, x_N^t)^T \in \mathbb{R}^{N\times L}$. To capture the temporal dynamics and provide context for prediction, we consider a historical window of $\tau$ time steps. The input data available at time $t$ for predicting the future resource requirements is the sequence of historical metric observations $\mathcal{X}_{t-\tau+1:t} = (\mathcal{X}_{t-\tau+1}, \ldots, \mathcal{X}_{t-1}, \mathcal{X}_t)$, which can be viewed as a tensor in $\mathbb{R}^{N\times L\times\tau}$.

The core problem is to determine the appropriate number of instance replicas for each service $v_i \in \mathcal{V}$ at the next time step $t+1$. This demand allocation relies on historical metric data $\mathcal{X}_{t-\tau+1:t}$ and multifaceted dependencies $\mathcal{A}$. Let $y_i^{t+1} \in \mathbb{R}_{\geq 0}$ be the predicted resource demand for service $v_i$ at time $t+1$. The output vector for all services is denoted by $\mathcal{Y}^{t+1} = (y_1^{t+1}, y_2^{t+1}, \ldots, y_N^{t+1})^T \in \mathbb{R}_{\geq 0}^N$. The formulation operates at the service level where individual instances do not evolve independently (i.e., horizontal scaling), which is a common setting in existing studies [5, 29, 43, 47]. The objective is to determine $\mathcal{Y}^{t+1}$ to simultaneously address two competing goals:

- *QoS Assurance:* Guarantee that the application meets its Service Level Agreements (SLAs), primarily concerning the end-to-end response time. This implies allocating enough instances to handle workload surges and maintain low latency.
- *Resource Utilization Optimization:* Achieve high utilization of the allocated computational resources across the microservice application. This aims to minimize resource waste and operational costs by avoiding over-provisioning.

Although both scaling dimensions can address this problem, we focus on horizontal scaling for practical considerations. First, vertical scaling in K8s often lacks support for "in-place" resource updates [3, 23], as modifying CPU or memory limits typically requires a container restart, disrupting request processing. Although K8s v1.35 recently graduated in-place Pod resource resize to GA [16], it still has known limitations such as restricted support for Pods with static CPU or memory policies and potential race conditions between the kubelet and scheduler. Second, vertical scaling introduces scheduling challenges [38], as increasing resource limits may cause instances to no longer fit on their node, triggering rescheduling and cascading evictions. Importantly, the core contribution of this work, i.e., the explicit modeling of heterogeneous service dependencies, is agnostic to the scaling mechanism. The model forecasts resource demand based on dependency propagation, which can be fulfilled by adding replicas or increasing per-instance allocations. We adopt horizontal scaling as a representative and operationally stable platform for validation. Extending our design to vertical scaling or combining both dimensions is a natural direction for future work.

# 3 Methodology

## 3.1 Overview

In this section, we present PANORAMA for microservice authoscaling, designed as a closed-loop control system. Fig. 2 shows the overall framework, which comprises four phrases: *performance monitoring*, *dependency mining*, *predictive autoscaling*, and *resource orchestration*. The performance monitoring phase periodically collects and stores workload-related metrics and microservice traces. The dependency mining phase then processes the traces to uncover fine-grained heterogeneous service dependencies. Next, the predictive autoscaling phase forecasts microservice resource needs using a hybrid attention mechanism and meta-learning strategy. Finally, the resource orchestration phase executes the predicted resource requirements to optimize resource allocation and service performance.

## 3.2 Performance Monitoring

System metrics provide real-time visibility into service performance, workload patterns, and resource utilization, which are essential for autoscaling decisions. However, obtaining fine-grained and accurate service-level metrics in real-world scenarios presents challenges, as cloud platforms may lack native support for such granular

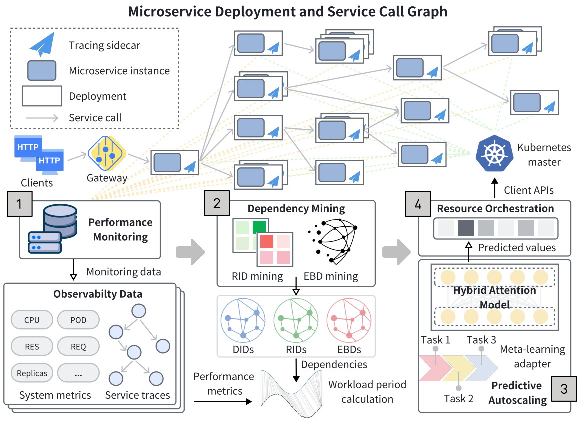


**Figure 2: Overview of the PANORAMA System Architecture**

monitoring. Traditional application-level instrumentation typically demands specialized developer expertise and can lead to intrusive code modifications or tight coupling with core microservice logic.

To enable comprehensive and transparent data collection, we deploy Envoy sidecar proxies into each microservice pod via the Istio service mesh, creating a dedicated data plane for observability. A *monitoring* component retrieves performance metrics (e.g., CPU utilization, memory usage, request latency, and request rate) from the sidecar proxies through Prometheus-compatible endpoints. Also, a *tracing* component leverages Jaeger (which is seamlessly integrated with Istio's data plane) to collect traces by intercepting inter-service communications at the proxy level. The traces are automatically propagated across service boundaries through standardized HTTP headers and gRPC metadata. Both metrics and distributed traces are then forwarded to a database backend for storage and subsequent processing. This architecture ensures that the data collection process remains completely transparent to the microservice developers, while providing the comprehensive observability data for effective dependency analysis and predictive autoscaling decisions.

## 3.3 Dependency Mining

Based on the collected telemetry data, this phase aims to construct a holistic service interaction graph, which captures different types of dependencies, i.e., DIDs, RIDs, and EBDs. Given the intricate and ever-evolving nature of microservice workloads, these dependencies are mined in real time to ensure that accurate and timely dependencies are available for autoscaling.

*3.3.1 Direct Invocation Dependencies.* DIDs are derived by parsing request traces collected in the performance monitoring phase to reconstruct call graphs. Each trace, representing an end-to-end request flow [9, 50], is composed of spans annotated with service identifiers, timestamps, etc. These spans encapsulate the operations of individual services throughout the lifecycle of a request. A DID is established from a parent service to a child service if a parent span initiates a call to a child span.

*3.3.2 Resource Interference Dependencies.* Identifying RIDs requires mapping logical services to their physical deployment topology to uncover potential resource contention among co-located instances. Using K8s as our reference environment, we establish this topology through a two-step extraction process.

First, we construct *service-to-instance* mappings (e.g., $UserService \rightarrow [Pod\text{-}A, Pod\text{-}B, Pod\text{-}C]$) by querying label selectors from *Deployment* and *Service* metadata. Subsequently, we build *instance-to-node* mappings (e.g., $[Pod\text{-}A, Pod\text{-}B, Pod\text{-}C] \rightarrow Node\text{-}X$) by inspecting the `spec.nodeName` attribute of each Pod. To maintain an accurate topology view during dynamic workload redistribution, we utilize the K8s Watch API to track real-time Pod lifecycle events. Instances co-residing on the same node are flagged as potential RIDs based on these mappings. However, simple co-location does not guarantee actual interference due to native isolation mechanisms like cgroups and CPU pinning. We address this discrepancy by dynamically validating and quantifying these structural dependencies. Specifically, we analyze their runtime workload characteristics by computing an attention score between co-located services utilizing their real-time performance metrics (see Sec. 3.4.2).

*3.3.3 Execution Blocking Dependencies.* The mining of EBDs employs a two-stage process. First, we infer *call order* relationships between services, which refers to the execution sequence of sibling microservices (i.e., services invoked by the same parent). Based on the call orders, we then identify deeper sequential constraints across various workflow execution paths, which constitute the EBDs.

**Call order construction.** We mine runtime traces to construct dynamic call orders, which is essentially a timestamp-based graph construction problem. We present only the basic idea and refer readers to [10, 30, 45] for more details. For each span of service $v_k$, we identify its child spans representing the sibling services (say $v_i$ and $v_j$) and sort them by start timestamps. In this sorted sequence, a pairwise call order $v_i \rightarrow v_j$ is considered if, for two chronologically adjacent spans $s_i$ and $s_j$ (corresponding to $v_i$ and $v_j$, respectively), they satisfy $s_j.start\_time \geq s_i.end\_time - \delta$. The small tolerance $\delta \geq 0$ accounts for potential clock skew between nodes. Within each parent service context, all such pairwise call orders are transitively merged to form *call order paths* of the parent $v_k$. For instance, merging $User\ validation \rightarrow Payment\ processing$ and $Payment\ processing \rightarrow Notification$ yields the path $User\ valida\text{–}tion \rightarrow Payment\ processing \rightarrow Notification$. This merging process is recursively applied until no more call orders can be merged.

**EBD mining.** Given a microservice $v_p$ that precedes $v_s$ within the same parent context, we identify the child services at the tail of $v_p$'s call order paths, whose completion is a prerequisite for $v_s$ to initiate. Each tail service is recursively traversed to explore its own tail services, forming a dependency chain that terminates at leaf services (those issuing no further calls) of the invocation graph rooted at $v_p$. An EBD is established from these tail leaf services to the direct successor of $v_p$, i.e., $v_s$, in the inferred call order. For illustration, consider the *Checkout* service in Fig. 1, where *Payment processing* precedes *Notification* in the call order. The path $[\cdots \rightarrow Trans.\ execution \rightarrow Shipping]$ represents one of the call order paths initiated by *Payment processing*. In this sequence, *Shipping* functions as the tail service. This is similar for *Inventory*, as *Payment processing* calls them concurrently. Since both *Shipping* and *Inventory* are tail services in distinct call order paths from *Payment processing*, they maintain an EBD with their common successor, namely, *Notification*. This logic extends to a finer granularity within the *Payment processing* workflow itself, where *Trans. execution* precedes both *Inventory* and *Shipping*. As *Trans. execution* is a leaf node, it is identified as its own tail service. This establishes an EBD from *Trans. execution* to its successors, indicating their execution is blocked until the transaction is complete.

In the above mining process, *false positives* may arise from clock skew or network jitter (mitigated by the tolerance parameter $\delta$) and *false negatives* may occur if certain execution paths remain unexercised during the observation window. The impact of both cases is tightly bounded by our design. The hybrid attention mechanism leverages real-time performance data to assign low attention scores to spurious dependencies, effectively filtering out false positives. In the presence of false negatives, PANORAMA gracefully falls back to utilizing DIDs alongside each service's inherent historical temporal patterns to ensure continuous autoscaler operation.

## 3.4 Predictive Autoscaling

After mining service dependencies, we introduce our model for dynamic workload forecasting and resource provisioning, as shown in Fig. 3. The architecture extends the principles of the Transformer model [42] to leverage heterogeneous service dependencies and handle multivariate time series data defined on a graph structure. The model is trained and adapted using a meta-learning strategy.

*3.4.1 Input Embedding.* The input metric data $\mathcal{X}_{t-\tau+1:t} \in \mathbb{R}^{N\times L\times\tau}$ are mapped into a $D$-dimensional embedding space. To enrich this initial representation with microservice contextual information, we incorporate both temporal and spatial positional encodings.

- **Temporal Positional Encoding (TPE).** To encode the sequential order within the historical window, a learnable positional representation $TPE_{t'} \in \mathbb{R}^D$ corresponding to time step $t' \in [t-\tau+1, t]$ is injected. This encoding explicitly captures the chronological position of each observation sequence.
- **Spatial Positional Encoding (SPE).** To capture the dynamic topological features of the microservice architecture, we introduce a spatial positional encoding $SPE_i \in \mathbb{R}^D$ for each service $v_i$. This encoding leverages the spectral properties of the invocation graph formed by the DIDs at time $t$, reflecting the microservices' real-time interaction patterns and traffic flow. Let $\mathcal{A}_t$ denote the adjacency matrix representing DID and $\mathcal{D}_t$ represent its diagonal degree matrix where $(\mathcal{D}_t)_{ii} = \sum_j (\mathcal{A}_t)_{ij}$. The Symmetric Normalized Laplacian $L_{sym,t}$ is computed using the identity matrix $I$ as:

$$L_{sym,t} = I - \mathcal{D}_t^{-1/2}\mathcal{A}_t\mathcal{D}_t^{-1/2} \tag{1}$$

  We extract the dynamic features $\phi_i \in \mathbb{R}^k$ from the eigenvectors associated with the $k$ smallest non-trivial eigenvalues of $L_{sym,t}$. The final encoding $SPE_i$ is produced via a dedicated linear transformation:

$$SPE_i = \mathcal{W}_s\phi_i + b_s \tag{2}$$

  where $\mathcal{W}_s \in \mathbb{R}^{D\times k}$ and $b_s \in \mathbb{R}^D$ constitute learnable weight and bias parameters. This spectral approach effectively embeds services into a continuous latent space where structural proximity and topological pathways are structurally preserved.

Let $e_i^{t'} \in \mathbb{R}^D$ denote the foundational $D$-dimensional representation of the raw features $x_i^{t'}$ after linear projection. The final input embedding for service $v_i$ at time step $t'$ integrates the temporal and spatial insights through element-wise addition:

$$h_i^{t'(0)} = e_i^{t'} + TPE_{t'} + SPE_i \tag{3}$$

These aggregated embeddings form the primary input tensor $\mathcal{H}^{(0)} \in \mathbb{R}^{N\times D\times\tau}$ for the subsequent encoder layers.

*3.4.2 Encoder Layers.* PANORAMA's prediction model consists of a stack of $K$ identical encoder layers. Each layer takes the output of the previous layer $\mathcal{H}^{(l-1)}$ and produces $\mathcal{H}^{(l)}$ through a Hybrid Dependency-Temporal Self-Attention mechanism and a position-wise Feed-Forward Network (FFN), interspersed with normalization and residual connections.

**Hybrid Dependency-Temporal Self-Attention**. The core of each encoder layer is a novel self-attention mechanism that decouples the standard global attention into independent heads, allowing each to capture the complex semantic relationships embedded within the temporal and various dependency dimensions. It comprises four sets of attention heads: one for temporal attention and three for the attentions of different dependency types. For a dependency attention head $\mathcal{A}^{(\kappa)}$ focusing on dependency type $\kappa$ ($\kappa \in \{did, rid, ebd\}$), the input $\mathcal{H} \in \mathbb{R}^{N\times D\times\tau}$ is projected into Queries ($\mathcal{Q}^{(\kappa,h)}$), Keys ($\mathcal{K}^{(\kappa,h)}$), and Values ($\mathcal{V}^{(\kappa,h)}$) for head $h$. Attention scores between nodes $i$ and $j$ at time $t'$ are computed as:

$$\alpha_{i,j}^{t'(\kappa,h)} = \frac{\mathcal{Q}_i^{t'(\kappa,h)} \cdot \mathcal{K}_j^{t'(\kappa,h)}}{\sqrt{D_h}} \tag{4}$$

where $D_h$ is the dimension of a single head. A dependency-based masking is applied such that $\alpha_{i,j}^{t'(\kappa,h)} = -\infty$ if $A_{ij}^{(\kappa)}$ indicates no dependency. The masked scores are normalized via softmax to obtain attention weights $\hat{\alpha}^{(\kappa,h)}$. This mechanism enables a service to disregard unrelated services and concentrate on those with dependencies. Consequently, the model is able to utilise the discrete dependency matrix to quantify the actual strength of these dependencies, which is represented by the attention scores $\alpha$. The utilization of these scores facilitates the enhancement of the model's capacity for effective and informed resource management decisions, underpinned by dynamic quantitative dependency information. The output for this head is the weighted sum of values:

$$O_i^{t'(\kappa,h)} = \sum_{j=1}^{N} \hat{\alpha}_{i,j}^{t'(\kappa,h)}\mathcal{V}_j^{t'(\kappa,h)} \tag{5}$$

Temporal attention heads operate similarly but compute attention scores across the $\tau$ dimension for each node $i$. The outputs from all heads are concatenated and linearly transformed.

**Feed-Forward Network and Residual Connections**. Following the attention mechanism, each encoder layer includes a position-wise FFN. To mitigate vanishing gradients and enhance training stability, residual connections [17] are integrated with layer normalization around both sub-layers. Additionally, stochastic depth [18] is employed as a regularization technique to prevent overfitting and enhance the model's generalization performance.

*3.4.3 Multi-scale Feature Aggregation and Output.* The outputs of all encoder layers are aggregated via residual connections to capture multi-scale features. The final aggregated representation is processed by a series of linear layers to produce the predicted

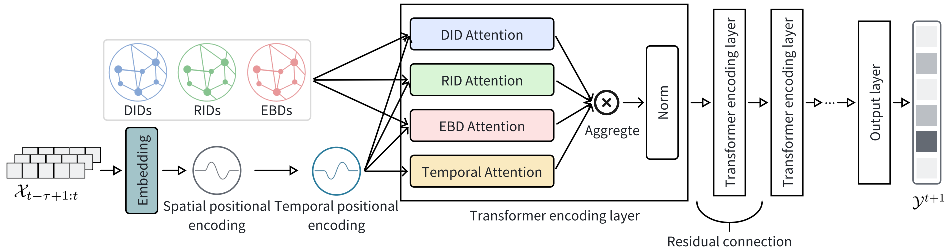


**Figure 3: The Architecture of PANORAMA's Prediction Model**

resource demand for each service at time $t+1$. We optimize our model by minimizing the L1 loss between the predicted demand vector $\mathcal{Y}^{t+1} \in \mathbb{R}^N_{\geq 0}$ and the ground-truth demand vector $\mathcal{Y}^{t+1}_{\text{true}} \in \mathbb{R}^N_{\geq 0}$. The loss function is defined as:

$$\mathcal{L}(\mathcal{Y}^{t+1}, \mathcal{Y}^{t+1}_{\text{true}}) = |\mathcal{Y}^{t+1} - \mathcal{Y}^{t+1}_{\text{true}}| \tag{6}$$

In the resource orchestration phase, this prediction vector, $\mathcal{Y}^{t+1} \in \mathbb{R}^N_{\geq 0}$, directly serves as the forecasted demand for all services.

*3.4.4 Meta-learning for Model Adaptation.* The operational environment of microservices is inherently dynamic and uncertain, characterized by continuous shifts in workload patterns and evolving service dependencies. Such heterogeneity can manifest as significant distributional drift relative to training data, severely degrading the predictive accuracy. Traditional approaches typically address this issue by retraining the model on newly observed data. However, given the rapid pace of change, frequent retraining incurs prohibitive time and maintenance costs. To endow PANORAMA with robust adaptive capacity, we design a meta-learning paradigm based on Model-Agnostic Meta-Learning (MAML) [11, 46], where we introduce the concept of *workload periods* as the core units for task division. A new workload period, i.e., a meta-learning task, is identified when the computed dissimilarity between consecutive segments of workloads [4] or dependencies (Jaccard Similarity) exceeds a predefined threshold. Each task encapsulates a particular operational scenario defined by its unique combination of request rate fluctuations, service interaction patterns, and underlying resource contention. In each workload period, the data collected is utilized to perform an adaptation step, fine-tuning the predictive model for the current operational context.

Unlike conventional fine-tuning or full retraining, the MAML framework meta-trains parameters across diverse workload tasks, yielding base weights amenable to rapid adaptation. Upon detecting distributional drift, PANORAMA adapts with only a few gradient steps on limited recent data, starting from near task-optimal parameters. Two contextual elements provide task-specific information during adaptation: spatial positional encoding (Sec. 3.4.1), capturing current invocation patterns, and real-time dependencies, integrated as structural priors through the attention mechanism.

## 3.5 Resource Orchestration

The final phase executes resource provisioning decisions. A key challenge is to prevent system oscillations, where overly frequent resource adjustments can trigger unnecessary scale-up and scale-down events, leading to instability and resource waste. To ensure stability, we implement two smoothing mechanisms. First, we establish a minimum time interval (six minutes) between consecutive scaling operations. Second, instead of making decisions based on a single predicted data point, we use the average predicted value over a future time window to smooth out minor fluctuations.

An *online scheduling* component takes the predicted resource demands, which are quantified as the number of instances in our experiments. It then incorporates potential static policies or constraints (e.g., minimum/maximum instance limits, deployment quotas). Then, it interacts with K8s via its client APIs to adjust the number of running instances for each microservice. Each scaling action modifies the replica count of the corresponding K8s Deployment, ensuring all instances within a service inherit identical resource specifications. This orchestration guarantees uniform allocation across replicas while dynamically adjusting aggregate processing capacity. Upon receiving the updated replica targets, the K8s control plane automatically provisions or terminates Pods across available worker nodes. This continuous adjustment physically reconfigures the runtime topology and compute capacity of the targeted microservices. These infrastructural modifications close the autonomous autoscaling loop. The resulting shifts in system state and operational metrics are subsequently captured by the performance monitoring phase to drive the next optimization cycle.

# 4 Experimental Evaluation

In this section, we conduct extensive experiments to evaluate the performance of PANORAMA. Our evaluation aims to answer the following research questions:

- **RQ1:** How effective is PANORAMA in autoscaling?
- **RQ2:** How adaptive is PANORAMA in autoscaling?
- **RQ3:** Are latent dependencies useful for autoscaling?
- **RQ4:** What is the performance overhead of PANORAMA?

## 4.1 Experimental Settings

*4.1.1 Benchmark Microservices.* The evaluation is conducted using four representative microservice benchmarks that exhibit characteristics of interactive and responsive applications. They use a variety of programming languages and frameworks (e.g., C#, Java, Python), which are widely used in microservice-related research [9, 24, 49] and provide diverse structural and workload complexities.

- *Online Boutique* [14] represents a cloud-native e-commerce prototype designed to showcase K8s and gRPC interactions. It models standard online retail operations such as merchandise browsing, cart management, and checkout workflows.
- *Social Network* [13] is an application featuring unidirectional follow relationships and multiple microservices communicating via Apache Thrift RPCs. It supports complex user interactions like multimedia posts and timeline reads. The resulting

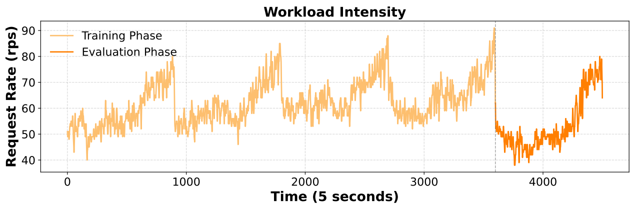


**Figure 4: Production Traffic from Alibaba's Traces**

complex service chains and diverse workloads make it highly suitable for scalability analysis.

- *Hotel Reservation* [13] implements a hotel reservation service built using Go and gRPC. It simulates user interactions involving transactional operations such as hotel querying, recommendation, and reservation. Its architecture involves interactions between multiple services to fulfill these user requests, making it a suitable testbed for autoscaling.
- *Train Ticket* [53] is a railway ticketing application comprising 41 microservices. These services are responsible for a wide array of functionalities, including user authentication, ticket booking, payment processing, and notification. Its extreme scale and functional density create an exceptionally challenging environment for system-wide optimization algorithms.

*4.1.2 Workload Generation.* To ensure our evaluation reflects realistic production conditions, we model our workload intensity based on a public microservice trace dataset from Alibaba [27]. These traces provide minute-level aggregate request rates per service interface, enabling us to extract our workload intensity profile. As illustrated in Fig. 4, the complete workload traffic consists of 4,500 data points and spans 6.25 hours. We employ Locust [26], an asynchronous workload generator, to replay this traffic profile against our benchmark applications. Each data point from the traffic, representing a target requests-per-second (rps) value, is used by Locust to govern the request generation for a five-second interval. At each second, Locust randomly invokes various service APIs based on the target rps. We partition this profile into a 5-hour training phase and a subsequent 1.25-hour evaluation phase.The left phase illustrates the traffic for training, showcasing clear periodic patterns and considerable fluctuations in rps over a long period, including sudden spikes and sharp drops in request rate that stress the autoscaler's responsiveness. The right phase presents the traffic for evaluation. It begins with an initial period of relative stability marked by minor fluctuations, followed by a distinct upward trend in the request rate. This profile thus represents a challenging scenario for assessing the adaptive capabilities of autoscaling mechanisms.

*4.1.3 Evaluation Metrics.* To validate the performance of different methods for microservice autoscaling, we use *instance count* and *response time* as the evaluation metrics, which represent the fundamental trade-off between resource cost and service performance. Instance count directly reflects the resource consumption and cost of and response time is the most critical indicator of service performance that impacts user experience, linked directly to QoS and SLAs. For robust and reliable comparison, we evaluate methods based on the *average instance count (AIC)* and the *average response time (ART)* observed over the experimental period.

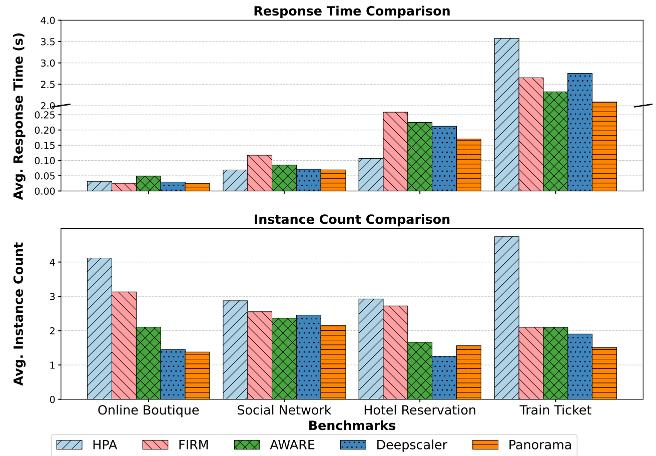


**Figure 5: Autoscaling Performance of Different Approaches**

Besides the trade-off between performance and cost, understanding the efficiency of resource usage and compliance with SLA constraints is equally important. Thus, we also calculate the *CPU utilization* of service instances and the *p95 response time* over the experiment period. It is a critical metric that reflects how intensively the allocated compute resources are being used. A higher average CPU utilization (without degrading performance) generally indicates more efficient resource usage and helps avoid resource over-provisioning. The p95 response time refers to the 95th percentile of response time among all requests. As a high-end latency metric, it captures system tail performance and is less susceptible to outliers compared to the maximum response time.

*4.1.4 Baseline Methods.* We compare PANORAMA's performance against several state-of-the-art methods, which represent diverse autoscaling paradigms, including traditional rule-based systems and proactive, learning-based approaches.

- *Horizontal Pod Autoscaling (HPA)* [22] serves as a fundamental rule-based baseline, reactively scaling instances when local resource metrics cross predefined thresholds. While widely used due to its simplicity, its strict reliance on rules limits its ability to proactively anticipate complex workload shifts.
- *DeepScaler* [29] is a autoscaling approach based on deep learning. It utilizes spatiotemporal graph neural networks and adaptive graph learning to capture service dependencies and accurately estimate resource needs for simultaneous scaling actions. It represents a state-of-the-art approach leveraging dependency-aware for microservice resource management.
- *FIRM* [36] is an reinforcement learning (RL)-based autoscaling approach that learns to adjust resources based on feedback. It finds services with abnormal response times through anomaly detection algorithms and adjusts multiple resources for the service through RL iterations, aiming to learn effective scaling policies from experience.
- *AWARE* [37] is another RL-based approach designed for cloud systems. It features rapid online learning mechanisms alongside a safe bootstrapping strategy to guarantee stable performance during both initial exploration and online serving phases.


**Figure 6: CPU Utilization of PANORAMA and DeepScaler**

*4.1.5 Experimental Configuration.* We conduct experiments on a 12-node K8s cluster. Each node is a VM with 8 dedicated vCPUs, 32 GB memory, and Ubuntu 22.04 LTS, distributed evenly across 6 physical hosts. We manage microservices as K8s Deployments with their default topologies, utilizing Prometheus for metrics and Jaeger for distributed tracing. Services operate in the K8s "burstable" QoS class with standard cgroup-based isolation. We avoid strict hardware partitioning to simulate realistic shared production environments. This configuration intentionally preserves the "noisy neighbor" contention to validate PANORAMA's explicit modeling of resource interference dependencies.

## 4.2 Experimental Results

*4.2.1 The Effectiveness of PANORAMA (RQ1).* To answer this question, we assess each approach on the selected benchmarks by measuring the service ART and AIC. As Fig. 5 shows, PANORAMA consistently achieves the best overall performance. On the simple Online Boutique benchmark, PANORAMA secures the lowest ART and AIC. Although the absolute ART differences are small on this benchmark, PANORAMA uses significantly fewer resources than all baselines, highlighting its resource efficiency. Similarly on Social Network, PANORAMA yields an ART comparable to HPA and DeepScaler while maintaining the lowest AIC. For Hotel Reservation, HPA provides the lowest ART through aggressive over-provisioning, whereas other baselines like FIRM suffer notable ART degradation. PANORAMA successfully balances this trade-off by achieving a low ART alongside a reduced AIC. Train Ticket presents the most challenging environment due to its massive scale and intricate dependencies. Here, PANORAMA distinctively outperforms all baselines, reducing AIC by 68% and improving ART by 42% compared to HPA. These results in a complex setting suggest the utility of PANORAMA's dependency-aware design for informed scaling decisions.

We also measure the CPU utilization of different approaches to assess their resource efficiency. Fig. 6 compares PANORAMA against DeepScaler, which is the strongest performing baseline, for two core microservices in the Train Ticket application. We can see that PANORAMA exhibits higher CPU utilization than DeepScaler for most of the observed period. Moreover, PANORAMA consistently maintains higher CPU utilization than DeepScaler on more than 70% of the Train Ticket's microservices. This shows its superior resource efficiency in effectively leveraging allocated resources. Beyond CPU utilization, we also measure the p95 response time of microservices, as shown in Fig. 7. For the selected core business services of Train Ticket, our method achieves lower p95 response times compared to all baselines. This enhanced tail-latency performance can be attributed to our method's dynamic dependency awareness, which enables timely and precise scale-out of critical services in response to sudden load spikes, thereby effectively mitigating SLA violations.

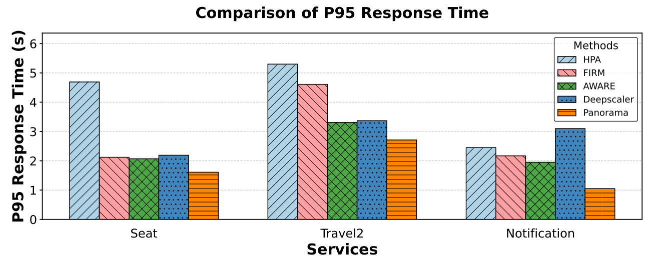


**Figure 7: P95 Response Time of Different Approaches**

*4.2.2 The Adaptability of PANORAMA (RQ2).* To answer this question, we evaluate each method's performance in a dynamic and changing environment, where the workloads are not included in the training data. We select the Train Ticket as our benchmark due to its close resemblance to real-world microservice deployments. For the workload, we select a set of complex request scenarios designed to stress the system's inter-service communication. This involves filtering for workload requests that flow through three or more distinct services within the application, ensuring a focus on non-trivial execution paths. Thus, the workload used for this validation represents a distinct workload period (Sec. 3.4.4) that significantly deviates from the historical data.

The results are presented in Fig. 8. While HPA demonstrates a marginally better ART than PANORAMA, it requires substantially more AIC. This can be attributed to HPA's inherent scaling mechanism, which, for such complex and dynamic workloads, often performs aggressive scale-out actions without adequately considering resource utilization. Consequently, HPA's AIC is approximately three times higher than that of PANORAMA, highlighting a significant trade-off in resource efficiency for a slight performance gain. On the other hand, PANORAMA outperforms all other baselines in both metrics. It achieves a lower ART than other baselines while consuming a comparable number of instances (AIC). Regarding RL-based methods such as FIRM and AWARE, their robust performance is largely attributable to their capacity for learning and adaptation. In contrast, DeepScaler records the highest ART, which indicates it is less effective at handling rapid workload changes. PANORAMA leverages RIDs and EBDs that capture the deep operational logic between services. When combined with meta-learning mechanism, it enables PANORAMA to effectively process and manage complex workloads within highly dynamic microservice environments.

*4.2.3 The Usefulness of Latent Dependencies (RQ3).* To validate the individual usefulness of latent dependencies, we conduct an ablation study based on Train Ticket, which features the most intricate dependency relationships. To assess the impact of the RIDs and EBDs, we configure different variants of PANORAMA that omit these components. As presented in Table 1, while "PANORAMA w/o RID & EBD" exhibits the best performance in terms of AIC, the absence of

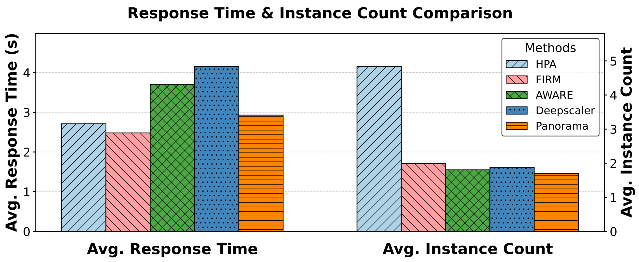


**Figure 8: Adaptability of Different Approaches**

RIDs and EBDs significantly degrades its ART, leading to a substantial increase of 51%. This degradation occurs because this variant can only react to superficial scaling needs of services with explicit call dependencies, overlooking other indirectly affected services that possess latent dependencies. This leads to bottlenecks in these unaddressed services, ultimately resulting in prolonged service response times and compromised service quality. Other experiments with "PANORAMA w/o RID" and "PANORAMA w/o EBD" reveal similar performance degradation patterns across both metrics.

We further demonstrate the usefulness of RIDs and EBDs through a case study on proactive bottleneck resolution. We focus on the *RoutePlan* microservice in the Train Ticket benchmark. This service orchestrates intensive travel planning computations and frequently becomes a system bottleneck during traffic surges. To eliminate the noise generated by mixed requests and deliberately trigger a bottleneck at *RoutePlan*, we apply a dedicated, upward-trending workload consisting exclusively of itinerary queries. Particularly, we exclude HPA due to its limitations in accurately considering resource utilization and its aggressive over-provisioning, which scales out a substantial number of instances for only marginal reductions in response time. Besides the ART of the RoutePlan service, we also calculate its average response time of requests (ARTR) to measure the system's end-to-end performance. The results are shown in Table 2. Particularly, the response times are higher than typical steady-state values since the system is intentionally initialized in a resource-constrained state. The reported metrics include the initial period of queue accumulation prior to autoscaler intervention. Once scaled, the system stabilizes at normal millisecond-level latency.

PANORAMA consistently outperforms all baselines in both metrics due to its latent dependency awareness. During the workload surge, PANORAMA leverages RIDs and EBDs to identify RoutePlan as a latent bottleneck, proactively scaling out *RoutePlan* and other related services. Conversely, the baseline methods lack this awareness and allocate additional instances to other services that exhibit more direct load signals, such as the upstream services that invoke *RoutePlan*. This leaves the root bottleneck unresolved. Furthermore, the increased throughput from the scaled upstream services overwhelms the unscaled *RoutePlan*, which severely exacerbates the bottleneck and degrades response times. These demonstrate that latent dependencies is essential for effective resource allocation.

Having demonstrated the critical impact of different dependencies, we now evaluate the dependency mining capabilities of PANORAMA against the leading method DeepScaler, which automatically infer latent relationships rather than explicitly modeling RIDs and EBDs. Fig. 9 shows heatmaps of the dependencies discovered by both methods on the Train Ticket, where each cell $(i, j)$ quantifies the dependency strength between the service $i$ and $j$. The comparison reveals a clear contrast in both the completeness and interpretability of the discovered dependencies. While DeepScaler effectively identifies DIDs and can indirectly infer some RIDs from correlated resource metrics, it largely fails to capture EBDs. This limitation arises because EBDs are rooted in workflow semantics, requiring a fine-grained understanding of execution order that cannot be derived from metric correlation alone. In contrast, PANORAMA's explicit mining process yields a far more comprehensive and semantically rich dependency graph, which allows it to proactively anticipate and mitigate performance degradation. Furthermore, these dependency insights provide actionable diagnostics for system optimization. For example, there are some dense columns in PANORAMA's RID heatmap, which identify services that act as resource hotspots. This enables operators to perform targeted interventions, such as adjusting resource allocations or re-scheduling services, to ensure more stable performance.

**Table 3: The Operational Overhead of PANORAMA**

| One-shot Operation | Overhead | | |
|---|---|---|---|
| | CPU (millicores) | Mem (MB) | Time (s) |
| Performance Monitoring | 558 ± 56 | 936 ± 18 | +∞ |
| Dependency Mining | 1148 ± 14 | 668 ± 7 | 21.5 ± 1.0 |
| Meta Learning | 970 ± 56 | 4355 ± 31 | 76.5 ± 2.4 |
| Resource Orchestration | 1711 ± 30 | 300 ± 10 | 20 ± 1.2 |

*4.2.4 The Overhead of PANORAMA (RQ4).* PANORAMA's overhead is a critical consideration for its practical deployment. We measure the resource consumption (CPU, memory) and execution time of its four core operations, including *performance monitoring*, *dependency mining*, *meta-learning*, and *resource orchestration*. We report CPU usage in millicores, where 1,000 millicores corresponds to one full CPU core. The results are summarized in Table 3. Performance monitoring relies on Prometheus and Jaeger to continuously collect metrics and traces. This persistent background process maintains a steady footprint of approximately 558 millicores of CPU and 936 MB of memory. Dependency mining periodically utilizes this data to extract RIDs and EBDs, requiring 1,148 millicores of CPU and 668 MB of memory for about 21.5 seconds per run. The meta-learning phase executes upon detecting new workload patterns to maintain adaptability. Fine-tuning model parameters consumes 970 millicores of CPU and 4355 MB of memory over roughly 76.5 seconds. Finally, resource orchestration performs lightweight model inference at each scaling interval, utilizing 1,711 millicores of CPU and 300 MB of memory. The observed 20-second orchestration time primarily results from our prototype serially issuing K8s API calls for Train Ticket's 41 microservices. A production deployment would parallelize these calls to substantially reduce latency. Our method demonstrates its practical significance by incurring modest overhead, making it suitable for production deployment.

## 4.3 Threats to Validity

The primary threat to *internal* validity lies in the implementation of our approach and the baselines. To mitigate this, we utilize author-provided implementations for the baselines and perform code reviews and testing of our own approach and experimental scripts.

**Table 1: Performance of PANORAMA Variants**

| Model Variant | ART (s) | AIC |
|---|---|---|
| PANORAMA | 2.08 | 1.44 |
| PANORAMA w/o RID | 2.95 | 1.43 |
| PANORAMA w/o EBD | 3.04 | 1.44 |
| PANORAMA w/o RID & EBD | 3.14 | 1.35 |

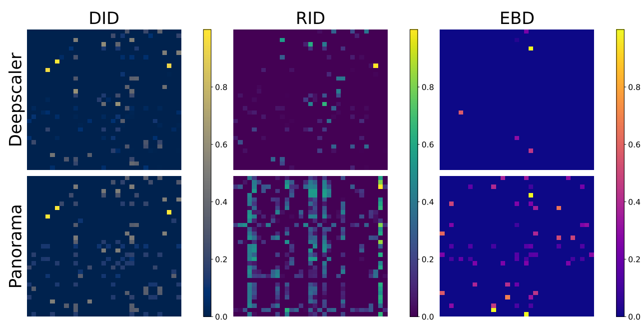


**Figure 9: Dependencies Mining Comparison**

Threats to *external* validity concern the generalizability of our findings. We mitigate this by using diverse microservice applications with varying scales and architectures. The method's adaptability is further tested with different workloads on Train Ticket. We also broaden our comparison by including both traditional rule-based and learning-based baselines. Moreover, our evaluation incorporate resource efficiency metrics like CPU utilization and P95 response time, providing a more comprehensive evaluation.

Threats to *construct* validity relate to whether our evaluation metrics accurately reflect the effectiveness of the approach. To alleviate it, we conduct a case study for fine-grained analysis and use visualization to show the effectiveness of the latent dependencies.

# 5 Related Work

## 5.1 Rule-based and Model-based Autoscaling

Rule-based autoscaling approaches, exemplified by K8s HPA [22] and commercial services like AWS Auto Scaling [1], Azure Autoscale [2], and Google Cloud Load Balancing [15], operate through predefined threshold-based mechanisms. While simple and widely adopted, these methods are reactive, treat services in isolation, and often struggle with the dynamic nature of microservice workloads.

Model-based approaches advance beyond thresholds by establishing mathematical relationships between workload and resource requirements, including queuing theory models [20, 41], time series models like ARMA [32], and workload prediction methods such as CloudScale [39] (FFT), AGILE [31] (wavelet transforms), and Autopilot [38] (exponential smoothing). However, these approaches often rely on strong assumptions about workload distributions and typically do not capture inter-service dependencies.

## 5.2 Deep Learning-based Autoscaling

Recent studies leverage deep learning techniques for more sophisticated, dependency-aware autoscaling. A prominent line of research utilizes Graph Neural Networks (GNNs) to model spatiotemporal relationships between microservices. For instance, DeepScaler [29] employs a spatiotemporal GNN with adaptive graph learning, iteratively refining an "affinity matrix" from the direct invocation graph to capture latent statistical correlations. Other GNN-based frameworks integrate system state into spatio-temporal GNNs [28] or focus on proactive allocation for tail latency SLOs [33]. While these methods move beyond static call graphs, the dependencies they learn are often black-box correlations lacking clear physical or logical semantics, making root cause diagnosis difficult.

**Table 2: A Case Study on the RoutePlan Service**

| Autoscaler | ART (s) | ARTR (s) |
|---|---|---|
| PANORAMA | 88.33 | 160.66 |
| FIRM | 114.87 | 183.60 |
| AWARE | 123.59 | 175.95 |
| DeepScaler | 120.11 | 167.51 |

Another category implicitly captures system dynamics without an explicit graph structure. For example, Sinan [51] treats the collective service state as an "image" and uses a a Convolutional Neural Network (CNN) to learn mappings from resource allocations to QoS. RL-based approaches, such as FIRM [36] and AWARE [37], learn scaling policies from environmental feedback. They are powerful but treat the system as a black box, with dependency understanding implicitly encoded in the learned policy or value function.

While these approaches have improved autoscaling, their dependency modeling relies on opaque correlations seeded on direct invocations, without distinguishing between fundamentally different interaction mechanisms. PANORAMA advances this field by moving from implicit correlation learning to explicit dependency modeling. Our core contribution is the systematic identification and mining of two semantically distinct categories of latent dependencies: resource interference dependencies, grounded in the physical deployment topology, and execution blocking dependencies, grounded in logical workflow semantics. By explicitly modeling heterogeneous dependencies, PANORAMA provides more accurate and interpretable autoscaling decisions.

# 6 Conclusion

In this paper, we present PANORAMA, a microservice autoscaling framework that addresses fundamental limitations in current approaches by explicitly modeling latent dependencies beyond direct service invocations. Our key contribution lies in the systematic formulation and identification of resource interference dependencies (RIDs) and execution blocking dependencies (EBDs), which capture critical yet previously invisible performance interactions in microservice architectures. PANORAMA employs hybrid attention mechanisms and meta-learning adaptation and operates as a closed-loop control system, enabling more accurate workload forecasting and proactive resource allocation decisions. Extensive experimental evaluation across diverse microservice benchmarks validates the effectiveness of PANORAMA, which consistently demonstrates superior performance compared to state-of-the-art baselines in terms of resource efficiency and service quality.

## Data Availability

Code and data available on https://doi.org/10.5281/zenodo.19249657.

## References


[1] AWS. 2022. AWS Auto Scaling Documentation. Retrieved May, 2025 from https://aws.amazon.com/autoscaling/

[2] Azure. 2022. Azure Autoscale. Retrieved May, 2025 from https://docs.microsoft.com/en-us/azure/azure-monitor/autoscale/

[3] Luciano Baresi, Davide Yi Xian Hu, Giovanni Quattrocchi, and Luca Terracciano. 2021. KOSMOS: Vertical and Horizontal Resource Autoscaling for Kubernetes. In *Service-Oriented Computing - 19th International Conference, ICSOC 2021, Virtual Event, November 22-25, 2021, Proceedings (Lecture Notes in Computer Science, Vol. 13121)*, Hakim Hacid, Odej Kao, Massimo Mecella, Naouel Moha, and Hye-young Paik (Eds.). Springer, 821–829. doi:10.1007/978-3-030-91431-8_59

[4] Donald J. Berndt and James Clifford. 1994. Using Dynamic Time Warping to Find Patterns in Time Series. In *Knowledge Discovery in Databases: Papers from the 1994 AAAI Workshop, Seattle, Washington, USA, July 1994. Technical Report WS-94-03*, Usama M. Fayyad and Ramasamy Uthurusamy (Eds.). AAAI Press, 359–370.

[5] Vivek M. Bhasi, Jashwant Raj Gunasekaran, Aakash Sharma, Mahmut Taylan Kandemir, and Chita R. Das. 2022. Cypress: input size-sensitive container provisioning and request scheduling for serverless platforms. In *Proceedings of the 13th Symposium on Cloud Computing, SoCC 2022, San Francisco, California, November 7-11, 2022*, Ada Gavrilovska, Deniz Altinbüken, and Carsten Binnig (Eds.). ACM, 257–272. doi:10.1145/3542929.3563464

[6] Tao Chen and Rami Bahsoon. 2017. Self-Adaptive and Online QoS Modeling for Cloud-Based Software Services. *IEEE Trans. Software Eng.* 43, 5 (2017), 453–475. doi:10.1109/TSE.2016.2608826

[7] Tao Chen, Rami Bahsoon, and Xin Yao. 2018. A Survey and Taxonomy of Self-Aware and Self-Adaptive Cloud Autoscaling Systems. *ACM Comput. Surv.* 51, 3 (2018), 61:1–61:40. doi:10.1145/3190507

[8] Zhuangbin Chen, Yu Kang, Liqun Li, Xu Zhang, Hongyu Zhang, Hui Xu, Yangfan Zhou, Li Yang, Jeffrey Sun, Zhangwei Xu, Yingnong Dang, Feng Gao, Pu Zhao, Bo Qiao, Qingwei Lin, Dongmei Zhang, and Michael R. Lyu. 2020. Towards intelligent incident management: why we need it and how we make it. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 1487–1497. doi:10.1145/3368089.3417055

[9] Zhuangbin Chen, Junsong Pu, and Zibin Zheng. 2025. Tracezip: Efficient Distributed Tracing via Trace Compression. *CoRR* abs/2502.06318 (2025). arXiv:2502.06318 doi:10.48550/ARXIV.2502.06318

[10] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM, 1285–1298. doi:10.1145/3133956.3134015

[11] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017 (Proceedings of Machine Learning Research, Vol. 70)*, Doina Precup and Yee Whye Teh (Eds.). PMLR, 1126–1135. http://proceedings.mlr.press/v70/finn17a.html

[12] Yu Gan, Mingyu Liang, Sundar Dev, David Lo, and Christina Delimitrou. 2021. Sage: practical and scalable ML-driven performance debugging in microservices. In *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Virtual Event, USA, April 19-23, 2021*, Tim Sherwood, Emery D. Berger, and Christos Kozyrakis (Eds.). ACM, 135–151. doi:10.1145/3445814.3446700

[13] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Cloud and IoT Microservices. *CoRR* abs/1905.11055 (2019). arXiv:1905.11055 http://arxiv.org/abs/1905.11055

[14] Google. [n. d.]. GitHub - GoogleCloudPlatform/microservices-demo: Sample cloud-first application with 10 microservices showcasing Kubernetes, Istio, and gRPC. — github.com. https://github.com/GoogleCloudPlatform/microservices-demo.

[15] Google. 2022. Google Cloud Load Balancing and Scaling. Retrieved May, 2025 from https://cloud.google.com/load-balancing/docs/autoscaling

[16] Natasha Sarkar (Google). 2025. Kubernetes 1.35: In-Place Pod Resize Graduates to Stable. Retrieved February, 2026 from https://kubernetes.io/blog/2025/12/19/kubernetes-v1-35-in-place-pod-resize-ga/

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 770–778. doi:10.1109/CVPR.2016.90

[18] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. 2016. Deep Networks with Stochastic Depth. In *Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part IV (Lecture Notes in Computer Science, Vol. 9908)*, Bastian Leibe, Jiri Matas, Nicu Sebe, and Max Welling (Eds.). Springer, 646–661. doi:10.1007/978-3-319-46493-0_39

[19] Darby Huye, Yuri Shkuro, and Raja R. Sambasivan. 2023. Lifting the veil on Meta's microservice architecture: Analyses of topology and request workflows. In *Proceedings of the 2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10-12, 2023*, Julia Lawall and Dan Williams (Eds.). USENIX Association, 419–432. https://www.usenix.org/conference/atc23/presentation/huye

[20] Dejun Jiang, Guillaume Pierre, and Chi-Hung Chi. 2010. Autonomous resource provisioning for multi-service web applications. In *Proceedings of the 19th International Conference on World Wide Web, WWW 2010, Raleigh, North Carolina, USA, April 26-30, 2010*, Michael Rappa, Paul Jones, Juliana Freire, and Soumen Chakrabarti (Eds.). ACM, 471–480. doi:10.1145/1772690.1772739

[21] Rouven Krebs. 2015. *Performance Isolation in Multi-Tenant Applications.* Ph. D. Dissertation. Karlsruhe Institute of Technology. http://digbib.ubka.uni-karlsruhe.de/volltexte/1000049479

[22] Kubernetes. 2025. Horizontal Pod Autoscaling. Retrieved May, 2025 from https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

[23] Kubernetes. 2025. Vertical Pod Autoscaling. Retrieved February, 2026 from https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/

[24] Bowen Li, Xin Peng, Qilin Xiang, Hanzhang Wang, Tao Xie, Jun Sun, and Xuanzhe Liu. 2022. Enjoy your observability: an industrial survey of microservice tracing and analysis. *Empir. Softw. Eng.* 27, 1 (2022), 25. doi:10.1007/S10664-021-10063-9

[25] Jianshu Liu, Shungeng Zhang, and Qingyang Wang. 2023. $\mu$ConAdapter: Reinforcement Learning-based Fast Concurrency Adaptation for Microservices in Cloud. In *Proceedings of the 2023 ACM Symposium on Cloud Computing, SoCC 2023, Santa Cruz, CA, USA, 30 October 2023 - 1 November 2023*. ACM, 427–442. doi:10.1145/3620678.3624980

[26] Locust. [n. d.]. Locust.io — locust.io. https://locust.io/.

[27] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Yu Ding, Jian He, and Chengzhong Xu. 2021. Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis. In *SoCC '21: ACM Symposium on Cloud Computing, Seattle, WA, USA, November 1 - 4, 2021*, Carlo Curino, Georgia Koutrika, and Ravi Netravali (Eds.). ACM, 412–426. doi:10.1145/3472883.3487003

[28] Yang Luo, Mohan Gao, Zhemeng Yu, Haoyuan Ge, Xiaofeng Gao, Tengwei Cai, and Guihai Chen. 2024. Integrating System State into Spatio Temporal Graph Neural Network for Microservice Workload Prediction. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, August 25-29, 2024*, Ricardo Baeza-Yates and Francesco Bonchi (Eds.). ACM, 5521–5531. doi:10.1145/3637528.3671508

[29] Chunyang Meng, Shijie Song, Haogang Tong, Maolin Pan, and Yang Yu. 2023. DeepScaler: Holistic Autoscaling for Microservices Based on Spatiotemporal GNN with Adaptive Graph Learning. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 53–65. doi:10.1109/ASE56229.2023.00038

[30] Animesh Nandi, Atri Mandal, Shubham Atreja, Gargi Banerjee Dasgupta, and Subhrajit Bhattacharya. 2016. Anomaly Detection Using Program Control Flow Graph Mining From Execution Logs. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, Balaji Krishnapuram, Mohak Shah, Alexander J. Smola, Charu C. Aggarwal, Dou Shen, and Rajeev Rastogi (Eds.). ACM, 215–224. doi:10.1145/2939672.2939712

[31] Hiep Nguyen, Zhiming Shen, Xiaohui Gu, Sethuraman Subbiah, and John Wilkes. 2013. AGILE: Elastic Distributed Resource Scaling for Infrastructure-as-a-Service. In *10th International Conference on Autonomic Computing, ICAC'13, San Jose, CA, USA, June 26-28, 2013*, Jeffrey O. Kephart, Calton Pu, and Xiaoyun Zhu (Eds.). USENIX Association, 69–82. https://www.usenix.org/conference/icac13/technical-sessions/presentation/nguyen

[32] Pradeep Padala, Kai-Yuan Hou, Kang G. Shin, Xiaoyun Zhu, Mustafa Uysal, Zhikui Wang, Sharad Singhal, and Arif Merchant. 2009. Automated control of multiple virtualized resources. In *Proceedings of the 2009 EuroSys Conference, Nuremberg, Germany, April 1-3, 2009*, Wolfgang Schröder-Preikschat, John Wilkes, and Rebecca Isaacs (Eds.). ACM, 13–26. doi:10.1145/1519065.1519068

[33] Jinwoo Park, Byungkwon Choi, Chunghan Lee, and Dongsu Han. 2021. GRAF: a graph neural network based proactive resource allocation framework for SLO-oriented microservices. In *CoNEXT '21: The 17th International Conference on emerging Networking EXperiments and Technologies, Virtual Event, Munich, Germany, December 7 - 10, 2021*, Georg Carle and Jörg Ott (Eds.). ACM, 154–167. doi:10.1145/3485983.3494866

[34] Jinwoo Park, Byungkwon Choi, Chunghan Lee, and Dongsu Han. 2024. Graph Neural Network-Based SLO-Aware Proactive Resource Autoscaling Framework for Microservices. *IEEE/ACM Trans. Netw.* 32, 4 (2024), 3331–3346. doi:10.1109/TNET.2024.3393427

[35] Xin Peng, Chenxi Zhang, Zhongyuan Zhao, Akasaka Isami, Xiaofeng Guo, and Yunna Cui. 2022. Trace analysis based microservice architecture measurement. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.). ACM, 1589–1599. doi:10.1145/3540250.3558951

[36] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. 2020. FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 805–825. https://www.usenix.org/conference/osdi20/presentation/qiu

[37] Haoran Qiu, Weichao Mao, Chen Wang, Hubertus Franke, Alaa Youssef, Zbigniew T. Kalbarczyk, Tamer Basar, and Ravishankar K. Iyer. 2023. AWARE: Automate Workload Autoscaling with Reinforcement Learning in Production Cloud Systems. In *Proceedings of the 2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10-12, 2023*, Julia Lawall and Dan Williams (Eds.). USENIX Association, 387–402. https://www.usenix.org/conference/atc23/presentation/qiu-haoran

[38] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. 2020. Autopilot: workload autoscaling at Google. In *EuroSys '20: Fifteenth EuroSys Conference 2020, Heraklion, Greece, April 27-30, 2020*, Angelos Bilas, Kostas Magoutis, Evangelos P. Markatos, Dejan Kostic, and Margo I. Seltzer (Eds.). ACM, 16:1–16:16. doi:10.1145/3342195.3387524

[39] Zhiming Shen, Sethuraman Subbiah, Xiaohui Gu, and John Wilkes. 2011. CloudScale: elastic resource scaling for multi-tenant cloud systems. In *ACM Symposium on Cloud Computing in conjunction with SOSP 2011, SOCC '11, Cascais, Portugal, October 26-28, 2011*, Jeffrey S. Chase and Amr El Abbadi (Eds.). ACM, 5. doi:10.1145/2038916.2038921

[40] Jiuchen Shi, Hang Zhang, Zhixin Tong, Quan Chen, Kaihua Fu, and Minyi Guo. 2023. Nodens: Enabling Resource Efficient and Fast QoS Recovery of Dynamic Microservice Applications in Datacenters. In *Proceedings of the 2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10-12, 2023*, Julia Lawall and Dan Williams (Eds.). USENIX Association, 403–417. https://www.usenix.org/conference/atc23/presentation/shi

[41] Jingwan Tong, Mingchang Wei, Maolin Pan, and Yang Yu. 2021. A Holistic Auto-Scaling Algorithm for Multi-Service Applications Based on Balanced Queuing Network. In *2021 IEEE International Conference on Web Services, ICWS 2021, Chicago, IL, USA, September 5-10, 2021*, Carl K. Chang, Ernesto Daminai, Jing Fan, Parisa Ghodous, Michael Maximilien, Zhongjie Wang, Robert Ward, and Jia Zhang (Eds.). IEEE, 531–540. doi:10.1109/ICWS53863.2021.00074

[42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008.

[43] Ziliang Wang, Shiyi Zhu, Jianguo Li, Wei Jiang, K. K. Ramakrishnan, Yangfei Zheng, Meng Yan, Xiaohong Zhang, and Alex X. Liu. 2022. DeepScaling: microservices autoscaling for stable CPU utilization in large scale cloud systems. In *Proceedings of the 13th Symposium on Cloud Computing, SoCC 2022, San Francisco, California, November 7-11, 2022*, Ada Gavrilovska, Deniz Altinbüken, and Carsten Binnig (Eds.). ACM, 16–30. doi:10.1145/3542929.3563469

[44] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. 2020. MicroRCA: Root Cause Localization of Performance Issues in Microservices. In *NOMS 2020 - IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, April 20-24, 2020*. IEEE, 1–9. doi:10.1109/NOMS47738.2020.9110353

[45] Yang Wu, Ang Chen, and Linh Thi Xuan Phan. 2019. Zeno: Diagnosing Performance Problems with Temporal Provenance. In *16th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2019, Boston, MA, February 26-28, 2019*, Jay R. Lorch and Minlan Yu (Eds.). USENIX Association, 395–420. https://www.usenix.org/conference/nsdi19/presentation/wu

[46] Yichen Wu, Long-Kai Huang, Renzhen Wang, Deyu Meng, and Ying Wei. 2024. Meta Continual Learning Revisited: Implicitly Enhancing Online Hessian Approximation via Variance Reduction. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://openreview.net/forum?id=TpD2aG1h0D

[47] Siqiao Xue, Chao Qu, Xiaoming Shi, Cong Liao, Shiyi Zhu, Xiaoyu Tan, Lintao Ma, Shiyu Wang, Shijun Wang, Yun Hu, Lei Lei, Yangfei Zheng, Jianguo Li, and James Zhang. 2022. A Meta Reinforcement Learning Approach for Predictive Autoscaling in the Cloud. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, Aidong Zhang and Huzefa Rangwala (Eds.). ACM, 4290–4299. doi:10.1145/3534678.3539063

[48] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 553–565. doi:10.1145/3611643.3616249

[49] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. DeepTraLog: Trace-Log Combined Microservice Anomaly Detection through Graph-based Deep Learning. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 623–634. doi:10.1145/3510003.3510180

[50] Lei Zhang, Zhiqiang Xie, Vaastav Anand, Ymir Vigfusson, and Jonathan Mace. 2023. The Benefit of Hindsight: Tracing Edge-Cases in Distributed Systems. In *20th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2023, Boston, MA, April 17-19, 2023*, Mahesh Balakrishnan and Manya Ghobadi (Eds.). USENIX Association, 321–339. https://www.usenix.org/conference/nsdi23/presentation/zhang-lei

[51] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G. Edward Suh, and Christina Delimitrou. 2021. Sinan: ML-based and QoS-aware resource management for cloud microservices. In *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Virtual Event, USA, April 19-23, 2021*, Tim Sherwood, Emery D. Berger, and Christos Kozyrakis (Eds.). ACM, 167–181. doi:10.1145/3445814.3446693

[52] Laiping Zhao, Yanan Yang, Yiming Li, Xian Zhou, and Keqiu Li. 2021. Understanding, predicting and scheduling serverless workloads under partial interference. In *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2021, St. Louis, Missouri, USA, November 14-19, 2021*, Bronis R. de Supinski, Mary W. Hall, and Todd Gamblin (Eds.). ACM, 22. doi:10.1145/3458817.3476215

[53] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Trans. Software Eng.* 47, 2 (2021), 243–260. doi:10.1109/TSE.2018.2887384


Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009